MEMORIA TÉCNICA Y METODOLOGÍA DE EJECUCIÓN

Anexo IV: Registro Forense de Operaciones y Procedimiento en Hashcat (CAI 1 - Parte 2)

Asignatura: Seguridad en Sistemas de Información e Internet | **Equipo Auditor:** Security Team 2 (ST-2)
Autores: Raimundo García Pérez, Konstantinos Peridis, Luis Alarcón Conradi
Entorno de Pruebas: Kali Linux (Virtualizado) | **Documento de Referencia:** ST-2-CAI1B

---

1. Entorno de Auditoría y Configuración del Sistema

Para evaluar la robustez del esquema HMAC empleado por la entidad bancaria sin comprometer los sistemas de producción, el equipo auditor reprodujo el escenario de ataque en un laboratorio aislado bajo la distribución especializada **Kali Linux**.

1.1. Arquitectura de Cómputo y Backend OpenCL

El entorno de pruebas no disponía de transferencia directa de tarjeta gráfica dedicada (*GPU passthrough*), por lo que la suite criptográfica **Hashcat (v7.1.2)** se configuró para operar exclusivamente sobre los núcleos de la CPU virtual (Intel Core i7-1065G7, 4 MCU) mediante la capa de abstracción OpenCL (backend PoCL). Se emplearon de forma sistemática los siguientes modificadores en todas las invocaciones:

- **-D 1** (*Device Type: CPU*): fuerza a Hashcat a utilizar exclusivamente los núcleos de procesamiento central como backend de cómputo.
- **-O** (*Optimized Kernels*): activa los núcleos de aceleración optimizados, adecuados para las longitudes de clave acotadas de este ejercicio (4 bytes).

2. Fase Previa: Normalización del Formato HMAC

El extracto remitido por la entidad financiera contenía tres mensajes de transferencia (cuenta origen, cuenta destino y cantidad, separados por un único espacio) junto a su código de autenticación HMAC codificado en **Base64**.

2.1. Deducción del Algoritmo de Resumen

Las tres cadenas Base64 presentaban una longitud constante de 44 caracteres terminados en relleno (=). Al decodificar el contenido a binario puro se obtienen exactamente 32 bytes (32×8 = 256 bits), lo que permite deducir de forma inequívoca el empleo de **HMAC-SHA-256** (modo Hashcat **-m 1450**, "HMAC-SHA256 (key = $pass)").

Para preparar las entradas requeridas por Hashcat se programó el script **hmac_to_hashcat.py** (adjunto en el Anexo I), que realiza:

- Decodificación de cada HMAC en Base64 a su representación hexadecimal de 64 caracteres.
- Verificación de que la longitud decodificada es de 32 bytes (256 bits).
- Generación del fichero canónico **hashes_hmac.txt** en formato hash_hex:mensaje, tal y como exige el modo 1450 (confirmado mediante hashcat -m 1450 --example-hashes).

```
# Ejecucion del script de normalizacion
python3 hmac_to_hashcat.py
```

2.2. Selección del Vector de Ataque

A diferencia de credenciales elegidas por usuarios (susceptibles de ataques por diccionario), la clave HMAC de la entidad es un secreto de **32 bits generado y distribuido por la propia entidad** en un dispositivo físico, por lo que no existe expectativa de estructura léxica. En consecuencia, el único vector de ataque aplicable con garantías de éxito es la **fuerza bruta exhaustiva** (**-a 3**) sobre el espacio completo de claves.

Se descartó la máscara ?a?a?a?a (charset ASCII imprimible, ~95^4 combinaciones) por restringir el espacio de búsqueda a claves textuales, cuando el enunciado especifica una clave de "tamaño 32 bits" sin garantía de que sean bytes ASCII. Se empleó en su lugar la máscara **?b?b?b?b**, que recorre el rango completo de cada byte (0x00-0xFF), agotando así exactamente el espacio real de 2^{32} = 4.294.967.296 claves posibles.

3. Ejecución del Ataque y Recuperación de Claves

3.1. Medición de la Capacidad de Cómputo (Benchmark)

```
hashcat -m 1450 -b -D 1 -O
```

Resultado del benchmark limpio (evidencia completa en **evidencias/benchmark.log**):

Métrica	Valor
Hash-Mode	1450 (HMAC-SHA256, key = $pass)
Dispositivo	CPU Intel Core i7-1065G7 (backend OpenCL/PoCL, sin GPU)
Velocidad medida	14.717,1 kH/s (~14,72 millones de intentos/segundo)

3.2. Ataque de Fuerza Bruta sobre los 3 Mensajes

```
hashcat -m 1450 -a 3 -D 1 -O hashes_hmac.txt ?b?b?b?b --outfile soluciones_hmac.txt
```

Hashcat evalúa simultáneamente los tres HMAC objetivo frente a cada candidato del keyspace, recorriendo el espacio de claves una única vez. El ataque recuperó las 3 claves en **9 minutos y 4 segundos**, habiendo explorado el **93,66 %** del espacio total (2^{32}) en el momento de la resolución completa, a una velocidad media sostenida de ~12,6-13,9 MH/s.

Mensaje	HMAC (hex, truncado)	Clave recuperada	Representación
34567893 987344 120	08bc6919...662fe599	20216767	" !gg" (ASCII imprimible)
34567894 986454 10	c1eedb4b...067827f22	99fa46a6	binario no imprimible
34537895 957769 110	6d3e367c...48ad18372	deadbeef	binario (patrón "deadbeef")

3.3. Verificación Criptográfica Independiente

Cada clave recuperada fue verificada de forma independiente mediante OpenSSL, recalculando el HMAC y confirmando su coincidencia exacta con el valor recibido de la entidad:

```
printf "34567893 987344 120" | openssl dgst -sha256 -mac HMAC -macopt hexkey:20216767
printf "34567894 986454 10"  | openssl dgst -sha256 -mac HMAC -macopt hexkey:99fa46a6
printf "34537895 957769 110" | openssl dgst -sha256 -mac HMAC -macopt hexkey:deadbeef
```

Los tres resultados coincidieron bit a bit con los HMAC originales del enunciado.

4. Consolidación Forense de Evidencias

Los resultados del fichero de estado (potfile) de Hashcat se volcaron a los registros de entrega mediante la opción --show, y se archivaron en la carpeta **evidencias/**:

```
hashcat -m 1450 --show hashes_hmac.txt
cat soluciones_hmac.txt
```

Ficheros de evidencia conservados: **evidencias/benchmark.log** (registro íntegro del benchmark), **evidencias/soluciones_hmac.txt** (salida cruda de Hashcat) y **evidencias/analisis_tiempos.log** (cálculo de tiempos y bits mínimos recomendados).

5. Lógica Algorítmica de los Scripts Auxiliares

El código fuente completo se adjunta en **hmac_to_hashcat.py** (Anexo I) y **analizar_hmac.py** (Anexo II). A continuación se resume su lógica procedimental.

5.1. Normalización de HMAC (hmac_to_hashcat.py)

```
ALGORITMO NormalizarHMACBase64:
  Entrada: Lista de mensajes con campo HMAC en Base64
  Para cada mensaje en la lista:
    1. Decodificar bytes_crudos = Base64_Decode(mensaje.HMAC_b64)
    2. Convertir digest_hex = Hexadecimal_Encode(bytes_crudos)
    3. Validar longitud: Longitud(digest_hex) == 64 (256 bits -> HMAC-SHA-256)
    4. Estructurar registro formato Hashcat modo 1450: "digest_hex:mensaje"
  Exportar registros a "hashes_hmac.txt" y "hashes_cai1b.json"
FIN ALGORITMO
```

5.2. Cálculo de Tiempos y Recomendación de Tamaño de Clave (analizar_hmac.py)

```
ALGORITMO CalcularTiemposYSeguridad(velocidad_H_s):
  espacio_32 = 2^32 ; espacio_medio = 2^31
  t_medio = espacio_medio / velocidad_H_s
  t_peor  = espacio_32 / velocidad_H_s
  vel_atacante = velocidad_H_s * 1000
  t_atacante_medio = espacio_medio / vel_atacante
  combinaciones_1_anio = vel_atacante * segundos_en_un_anio
  bits_minimos = techo(log2(combinaciones_1_anio))
  Mostrar tiempos, velocidad de atacante y bits minimos recomendados
FIN ALGORITMO
```